

Intel e ARM assembly - 05/06/2026

Nota 1 Versioni compilatore

Negli esercizi sono stati usate le seguenti versioni dei compilatori:

- **Intel:** x86-64 gcc 15.2
- **ARM:** ARM GCC 15.2.0

Entrambe queste con ottimizzazione “-O1”, dunque se voleste usare “godbolt” per verificare le vostre soluzioni, assicuratevi di selezionare queste versioni dei compilatori per evitare discrepanze dovute a ottimizzazioni o cambiamenti nelle convenzioni di chiamata. Inoltre assicuratevi che nelle impostazioni di “output” sia selezionata la sola “Demangle identifiers” e le altre opzioni siano disabilitate, in modo da avere un output più pulito e facilmente leggibile.

1 Intel x86-64

Esercizio 1

2.75 punti

Quale è una caratteristica distintiva dell'architettura Intel x86-64?

- a) Per usare i registri a 64 bit è necessario utilizzare un prefisso speciale in ogni istruzione (nel nome del registro),
- b) L'architettura supporta solo 8 registri generali a 64 bit,
- c) Non sono presenti istruzioni senza che un registro usato come argomento venga modificato
- d) I flag vengono modificati solo dalle istruzioni “test” e “cmp”

Soluzione: L'architettura Intel x86-64 supporta 16 registri generali a 64 bit, ma per accedere alla parte a 64 bit di questi registri è necessario utilizzare un prefisso speciale (ad esempio, `%rax` invece di `%eax`). L'unica istruzione che non modifica un registro già usato come argomento è `test`, oppure anche `leal` che però non modifica i flag. Tutte le altre istruzioni che operano su registri modificano almeno uno dei registri usati come argomento e/o i flag. Infine i flag vengono modificati da molte istruzioni, non solo da `test` e `cmp`, ma anche da operazioni aritmetiche, logiche, di shift, ecc. Di conseguenza, la prima opzione è l'unica corretta.

Esercizio 2

2.75 punti

L'indirizzamento basato su registri in Intel x86-64 consente di accedere alla memoria con 4 argomenti, quali sono e come viene calcolato l'indirizzo finale?

- a) Base, Index, Scale, Displacement; Indirizzo = $\text{Base} + (\text{Index} * \text{Scale}) + \text{Displacement}$
- b) Base, Index, Scale, Displacement; Indirizzo = $\text{Base} + \text{Index} + (\text{Scale} * \text{Displacement})$
- c) Base, Index, Scale, Displacement; Indirizzo = $(\text{Base} * \text{Scale}) + \text{Index} + \text{Displacement}$
- d) Base, Index, Scale, Displacement; Indirizzo = $(\text{Base} * \text{Index}) + (\text{Scale} * \text{Displacement})$

Soluzione: L'indirizzamento basato su registri in Intel x86-64 utilizza quattro argomenti: Base, Index, Scale e Displacement. L'indirizzo finale viene calcolato come: $\text{Indirizzo} = \text{Base} + (\text{Index} * \text{Scale}) + \text{Displacement}$. La Base è un registro che contiene un indirizzo di base, l'Index è un registro che contiene un indice, la Scale è un fattore di scala (1, 2, 4 o 8) che moltiplica l'Index, e il Displacement è una costante che viene aggiunta al risultato. Di conseguenza, la prima opzione è corretta.

Esercizio 3

2.75 punti

Riporta quale di queste istruzioni è corretta per completare la traduzione del seguente frammento di codice in Intel x86-64:

```
1 long long int mystery(long long int i, long long int j, long long int f
  , long long int g) {
2     int z = 0;
3     while(z < g){
4         f = f + j;
5         z = z + 1;
6     }
7     return f;
8 }
```

Traduzione Intel x86-64:

```
1 mystery:
2     testq    %rcx, %rcx
3     jle      .L4
4     movl     $0, %edi
5 .L3:
6     movq     %rdi, %r8
7     addq     $1, %rdi
8     cmpq     %rdi, %rcx
9     jne      .L3
10    addq     %rsi, %rdx
11    imulq    %rsi, %r8
12    X1
13    ret
14 .L4:
15    movq     %rdx, %rax
16    ret
```

Quale delle seguenti istruzioni è corretta per completare la traduzione?

- a) `leaq (%rdx,%rsi), %rax`
- b) `leaq (%rdx,%rdi), %rax`
- c) `leaq (%rdx,%r8), %rax`
- d) `leaq (%rdx,%rcx), %rax`
- e) Nessuna delle precedenti

Soluzione: Procediamo analizzando cosa viene tradotto nel frammento di codice assembly. I parametri della funzione C sono passati nei registri secondo le convenzioni di chiamata: `%rdi` (i), `%rsi` (j), `%rdx` (f) e `%rcx` (g). Come per molti programmi nel quale è presente un ciclo verifichiamo se la condizione di uscita è già verificata all'inizio della funzione (test su `%rcx`, facendo l'*and* bit a bit verifichiamo se il valore è zero o negativo, in tal caso saltiamo alla fine).

Il ciclo è compreso tra l'etichetta `.L3` e il salto condizionale `jne .L3`, ma prima notiamo l'inizializzazione del registro `%edi` a 0, che rappresenta la variabile `z` del codice C. All'interno del ciclo, `%rdi` viene incrementato di 1 (corrispondente a $z = z + 1$), e viene confrontato con `%rcx` (corrispondente a `g`) per verificare se il ciclo deve continuare. Al raggiungimento della condizione di uscita, ($z = g$ data da `jne .L3`), il ciclo termina e si procede con l'istruzione `addq %rsi, %rdx`, che corrisponde a $f = f + j$ in quanto in `f` è memorizzato il valore di `rdx` (convenzioni di chiamata), e in `j` è memorizzato il valore di `rsi`, dunque $\%rdx = j + f$.

Successivamente notiamo che viene moltiplicato il valore di `j` (in `rsi`) per il numero di iterazioni del ciclo, meno l'ultima iterazione di uscita, questo lo notiamo in quanto viene eseguita una `movq %rdi, %r8` prima dell'incremento di `z` e del confronto con `g`, dunque in `r8` è memorizzato $g - 1$, dunque dopo la `imulq %rsi, %r8` abbiamo $r8 = j \cdot (g - 1)$.

Non ci sono altre istruzioni prima del ritorno il che significa che deve essere impostato il valore di ritorno in `%rax`, questo non ci permette ancora di escludere le opzioni. Tuttavia al momento abbiamo $r8 = j \cdot (g - 1)$ e $rdx = f + j$, la nostra funzione deve ritornare $f + j \cdot g$, dunque l'istruzione mancante deve essere `leaq (%rdx,%r8), %rax`, che calcola $(f + j) + (j \cdot (g - 1)) = f + j \cdot g$ e lo memorizza in `%rax` come valore di ritorno.

Risposta giusta: c)

Esercizio 4

2.75 punti

Quale di queste istruzioni è corretta per completare la traduzione del seguente frammento di codice in Intel x86-64:

```
1 void vectOps(short* a, short* b, short* c, short n){
2     short i=0;
3     while(i<n){
4         if(b[i]>c[i])
5             a[i++] = b[i]-c[i];
6         else
7             a[i++] = c[i]-b[i];
8     }
9 }
```

Traduzione Intel x86-64:

```
1 vectOps:
2     movq    %rdi, %r10
3     movq    %rsi, %r11
4     testw   %cx, %cx
5     jle     .L1
6     movswq  %cx, %rcx
7     movl    $1, %eax
8     jmp     .L5
9 .L6:
10    movq    %rsi, %rax
11 .L5:
12    movzwl  -2(%r11,%rax,2), %r8d
13    movzwl  -2(%rdx,%rax,2), %edi
14    movl    %r8d, %r9d
15    subl    %edi, %r9d
16    movl    %edi, %esi
17    subl    %r8d, %esi
```

```

18      cmpw    %di, %r8w
19      cmovg   %r9d, %esi
20      X1
21      leaq    1(%rax), %rsi
22      cmpq    %rcx, %rax
23      jne     .L6
24  .L1:
25      ret

```

Quale delle seguenti istruzioni è corretta per completare la traduzione?

- | | |
|---|--|
| a) <code>movw %si, -2(%rdi,%rax,2)</code> | b) <code>movw %si, (%r10,%rax,2)</code> |
| c) <code>movw %si, -2(%r10,%rax,2)</code> | d) <code>movw %r9w, -2(%r10,%rax,2)</code> |

Soluzione: Partiamo col tenerci traccia dei parametri della funzione C, che sono passati nei registri secondo le convenzioni di chiamata: `%rdi` (a), `%rsi` (b), `%rdx` (c) e `%rcx` (n). Tutti questi parametri sono classificati come puntatori a short, dunque ogni elemento degli array occupa 2 byte. Noriamo che viene eseguito un primo spostamento degli indirizzi di base di a e b nei registri `%r10` e `%r11`, poi viene eseguito l'oramai aspettato test su n (in `%cx`) per verificare se il ciclo deve essere eseguito, e se sì viene esteso a 64 bit all'interno di se stesso (`movswq %cx, %rcx`, *MOVE Signed Word to Quadword*), poi viene inserito il valore 1 in `%eax`, quindi ipotizziamo che questo sarà il nostro indice *i*, ma memorizzato come *i* + 1.

Il programma poi entra nel ciclo la prima volta a partire dal .L5 poi sempre da .L6. Le prime istruzioni che vengono eseguite sono il caricamento di `b[i]` e `c[i]` nei registri `%r8d` e `%edi` rispettivamente, possiamo dire ciò in quanto i registri che vengono usati come base sono `%r11` (dove è stato salvato l'indirizzo di b) e `%rdx` (dove è stato salvato l'indirizzo di c), e l'offset è dato da $i + 1 \cdot 2$ (dato che ogni elemento occupa 2 byte) però in questo contesto non viene caricato l'elemento *i* + 1 ma l'elemento *i*, in quanto l'indice è memorizzato come *i* + 1, ma viene eseguito un offset di -2 byte, dunque $i + 1 - 1 = i$. Successivamente salviamo il valore di `b[i]` in `%r9d` e calcoliamo sia `b[i] - c[i]` su quest'ultimo registro, facciamo la stessa cosa per `c[i] - b[i]` ma questa volta memorizzando il risultato in `%esi`. Eseguiamo ora il confronto tra `b[i]` (in `%r8w`) e `c[i]` (in `%di`), notare come sebbene le operazioni di spostamento e sottrazione vengano eseguite su tutti i 32 bit dei registri, il confronto viene eseguito solo sui 16 bit inferiori, questo è dovuto al fatto che stiamo lavorando con short, dunque con dati a 16 bit. L'istruzione seguente è una `cmovg` (*Conditional MOVE if Greater*), che se la condizione è verificata (ovvero se `b[i] > c[i]`, dato che il confronto è stato fatto tra `b[i]` e `c[i]`), allora esegue l'operazione `%esi = %r9d`, altrimenti lascia invariato il valore di `%esi` (che contiene il risultato di `c[i] - b[i]`).

Le istruzioni dopo quella mancante servono ad incrementare l'indice *i* (memorizzato come *i* + 1 in `%rsi`) preservandone quindi il contenuto e a confrontarlo con *n* (in `%rcx` per convenzioni di chiamata) per verificare se il ciclo deve continuare, in caso affermativo si passa alla *label* .L6 che riposiziona il valore di *i* + 1 attuale nel registro `%rax` e il ciclo prosegue.

L'istruzione mancante in tutto questo è la memorizzazione del risultato in `a[i]`, l'indirizzo di a è stato salvato in `%r10`, l'indice *i* + 1 è memorizzato in `%rsi`, stiamo usando **short** quindi ogni elemento occupa 2 byte, dunque l'offset è dato da $i + 1 \cdot 2$, ma in realtà dobbiamo memorizzare il risultato in *i* **in quanto viene usato un operatore di post incremento e non pre incremento** e quindi dobbiamo eseguire un offset di -2 byte, dunque l'istruzione corretta è `movw %si, -2(%r10,%rax,2)`, che memorizza il risultato ottenuto nel registro *word* `%si` (che

contiene il risultato di $b[i] - c[i]$ o $c[i] - b[i]$ a seconda del confronto) nell'indirizzo di $a[i]$ (dato da $\%r10 + i + 1 \cdot 2 - 2$ byte).

Risposta giusta: b)

Esercizio 5

2.75 punti

Quale di queste istruzioni è corretta per completare la traduzione del seguente frammento di codice in Intel x86-64:

```
1 long long int supreamSum(long long int* a, long long int* b, short n,
  short m){
2     short i=0;
3     long long int sum=0;
4     while(i<n && i<m){
5         sum+= a[i]*b[i++];
6     }
7     return sum;
8 }
```

Traduzione Intel x86-64:

```
1 supreamSum:
2     movq    %rdi, %r8
3     cmpw    %dx, %cx
4     cmovg   %edx, %ecx
5     testw   %cx, %cx
6     jle     .L4
7     movswq  %cx, %rcx
8     X1
9     movl    $0, %eax
10    movl    $0, %ecx
11 .L3:
12    movq    (%r8,%rax), %rdx
13    X2
14    addq    %rdx, %rcx
15    addq    $8, %rax
16    cmpq    %rdi, %rax
17    jne     .L3
18 .L1:
19    movq    %rcx, %rax
20    ret
21 .L4:
22    movl    $0, %ecx
23    jmp     .L1
```

Quale delle seguenti opzioni è corretta per completare la traduzione?

- a) X1: `leaq 0(,%rcx,8), %rdi`; X2: `imulq (%rsi,%rax), %rdx`
- b) X1: `leaq 0(,%rcx,4), %rdi`; X2: `imull (%rsi,%rax), %edx`
- c) X1: `leaq 0(,%rcx,8), %rdi`; X2: `imulq (%r8,%rax), %rdx`
- d) X1: `leaq 0(,%rcx,2), %rdi`; X2: `imulq (%rsi,%rax), %rdx`
- e) Nessuna delle precedenti

Soluzione: Analizziamo il codice; per le convenzioni di chiamata i parametri sono passati nei registri `%rdi` (a), `%rsi` (b), `%dx` (n) e `%cx` (m). Notiamo che la convenzione perde subito la sua completa efficacia infatti n viene spostato subito in `%r8`. Anche in questo caso la prima cosa da verificare è la possibilità di eseguire il ciclo, in questo caso la condizione di uscita algebricamente è il minimo tra n e m , dunque il ciclo viene eseguito se $i < \min(n, m)$, questo è verificato con le istruzioni `cmpw %dx, %cx` (confronto tra n e m) e `cmovg %edx, %ecx` (se $n > m$ allora `%ecx` = n , altrimenti `%ecx` = m per convenzioni di chiamata). In questo modo `%ecx` contiene il minimo tra n e m , dunque la condizione di uscita del ciclo è verificata con un semplice test su `%ecx` (test su $\min(n, m)$) e un salto condizionale. Se il ciclo può essere eseguito, allora viene esteso a 64 bit all'interno di se stesso (`movswq %cx, %rcx`, *MOVE Signed Word to Quadword*), poi viene inserito il valore 0 in `%eax` (dove sarà memorizzato il risultato finale per convenzione di chiamata) e in `%ecx`.

Tralasciamo per ora l'istruzione mancante, ma teniamo a mente che in `%rcx` è memorizzato l'indice minimo tra n e m .

Notiamo grazie all'istruzione di salto condizionato `jne .L3` che il ciclo è compreso tra `.L3` e questo salto. Ricapitoliamo quello che ci aspettiamo di vedere all'interno del ciclo:

- Caricamento di qualche registro con il valore di `a[i]`
- Caricamento di qualche registro con il valore di `b[i]` (e non $i + 1$ in quanto l'incremento è postfisso)
- Moltiplicazione tra questi due valori
- Somma del risultato alla variabile `sum`
- Incremento dell'indice i
- Confronto tra i e $\min(n, m)$ per verificare se il ciclo deve continuare

In tutto questo ricordiamo però che a differenza di architetture come RISC-V in Intel x86-64 possiamo combinare uno o più di questi passaggi in un'unica istruzione.

La prima istruzione di questo ciclo carica nel registro `%rdx` il valore di `a[i]`, visto che l'indirizzo di `a` è stato salvato in `%r8` allora in `%rax` ci deve essere salvato il valore dell'indice i **già moltiplicato per 8** (dato che ogni elemento di `a` occupa 8 byte).

Anche in questo caso tornerò sull'istruzione mancante in un secondo momento.

Successivamente viene eseguita un'istruzione di addizione tra `%rdx` e `%rcx`, e c'è l'incremento del registro `%rax` di 8 che ci conferma che in questo è memorizzato l'offset già moltiplicato per 8, dunque $i \cdot 8$. Viene quindi eseguito un confronto tra `%rax` e `%rdi`, questo confronto è la nostra condizione per rimanere nel ciclo, ciò implica che nel registro `%rdi` il valore di fine deve essere già moltiplicato per 8 in quanto il valore del registro `%rax` è già moltiplicato per 8, questo significa che l'istruzione mancante in X1 è la moltiplicazione del valore di indice minimo memorizzato in `%rcx` per 8 e il salvataggio del risultato in `%rdi`, dunque l'istruzione mancante è `leaq 0(%rcx,8), %rdi`, che calcola $\min(n, m) \cdot 8$ e lo memorizza in `%rdi` come valore di fine per il ciclo. **Escludiamo quindi b) e d).**

A questo punto dalla nostra "lista" di passaggi che ci aspettiamo di vedere all'interno del ciclo, abbiamo visto con certezza il caricamento in `%rdx` di `a[i]`, l'incremento dell'indice i (dato dall'incremento di `%rax` di 8), e il confronto tra i e $\min(n, m)$ (dato dal confronto tra `%rax` e `%rdi`). Inoltre abbiamo visto la somma `addq %rdx, %rcx` che possiamo dedurre essere la

somma del risultato della moltiplicazione e il valore attuale di `sum` questo in quanto, prima di ritornare il controllo con `ret`, il valore del registro `%rcx` viene spostato in `%rax` che è il registro che deve contenere il valore di ritorno per convenzioni di chiamata, dunque `%rcx` contiene il valore di `sum` aggiornato.

Come conseguenza le ultime istruzioni che mancano sono il caricamento di `b[i]` e la moltiplicazione tra `a[i]` (in `%rdx`) e `b[i]`, questa istruzione in x86-64 è scrivibile in una unica istruzione con operando di memoria. Questa istruzione ha come secondo argomento/destinazione `%rdx` vista la somma successiva e il valore di `a[i]` che è già memorizzato in questo registro. Il primo argomento/sorgente è dato da `b[i]`, l'indirizzo di `b` è stato salvato in `%rsi` come da *calling convention* e l'indice `i` è memorizzato come `i * 8` in `%rax`, **dunque l'istruzione mancante è `imulq (%rsi,%rax), %rdx`**, che moltiplica il valore di `b[i]` (dato da `(%rsi,%rax)`) per il valore di `a[i]` (in `%rdx`) e memorizza il risultato in `%rdx` sovrascrivendo il valore di `a[i]` con il risultato della moltiplicazione, che poi viene sommato a `sum` (in `%rcx`).

Risposta giusta: a)

Esercizio 6

2.75 punti

Nota 2 Esercizio bonus

Il presente esercizio è stato proposto al tutorato svolto in data 06/05/2026, successivamente mi sono reso conto che la difficoltà di questo esercizio è data principalmente da istruzioni Intel x86-64 che non sono state trattate durante il corso e che non sono richieste per la comprensione del corso stesso.

Non avendo trattato queste istruzioni durante il corso, non mi sembra corretto chiedere agli studenti di risolvere questo esercizio, tuttavia, dato che è un esercizio interessante e che può essere risolto con le conoscenze acquisite durante il corso, ho deciso di lasciarlo come esercizio bonus per chi volesse cimentarsi nella sua risoluzione.

Quale di queste istruzioni è corretta per completare la traduzione del seguente frammento di codice in Intel x86-64:

```
1 void pointers(int* test, int a, int b, int c){
2     int d = 0;
3     while(d < c){
4         if(d % a == 0){
5             d++;
6         }else{
7             d+=2;
8         }
9     }
10    *test = d;
11 }
```

Traduzione Intel x86-64:

```
1 pointers:
2     testl    %ecx, %ecx
3     jle      .L6
4     movl     $0, %r8d
5 .L5:
6     movl     %r8d, %eax
```

```

7      cltd
8      idivl    %esi
9      X1
10     X2
11     cmpl     %ecx, %r8d
12     jl       .L5
13 .L2:
14     movl     %r8d, (%rdi)
15     ret
16 .L6:
17     movl     $0, %r8d
18     jmp      .L2

```

Quale delle seguenti opzioni è corretta per completare la traduzione?

- a) X1: `cmpl $1, %edx`; X2: `sbb1 $-2, %r8d`
- b) X1: `cmpl $0, %edx`; X2: `sbb1 $-2, %r8d`
- c) X1: `cmpl $1, %edx`; X2: `adcl $2, %r8d`
- d) X1: `testl %edx, %edx`; X2: `sbb1 $-1, %r8d`

Soluzione: Analizziamo il codice; per le convenzioni di chiamata i parametri sono passati nei registri `%rdi` (test), `%esi` (a), `%edx` (b) e `%ecx` (c). La funzione implementa un ciclo `while` in cui la variabile `d` (salvata nel registro `%r8d`) viene incrementata condizionalmente in base al resto della divisione `d % a`.

All'interno del ciclo `.L5`, il compilatore utilizza l'istruzione `idivl %esi` per calcolare la divisione tra `d` (in `%eax`) e `a`. Il resto della divisione viene memorizzato nel registro `%edx`. Per evitare salti condizionali costosi derivanti dall'istruzione `if`, il compilatore utilizza una tecnica basata sulla gestione del Carry Flag (CF).

L'istruzione in `X1` è `cmpl $1, %edx`, che confronta il resto con 1. Se il resto è 0 (`d % a == 0`), l'operazione $0 - 1$ imposta il Carry Flag a 1. In caso contrario, il resto è ≥ 1 e il Carry Flag è 0. L'istruzione in `X2` è `sbb1 $-2, %r8d`, che esegue l'operazione: $dest = dest - (sorgente + CF)$. Nel nostro caso: $d = d - (-2 + CF)$, ovvero $d = d + 2 - CF$.

- Se $CF = 1$ (resto 0), allora $d = d + 2 - 1 = d + 1$, corrispondente a `d++`.
- Se $CF = 0$ (resto $\neq 0$), allora $d = d + 2 - 0 = d + 2$, corrispondente a `d += 2`.

Di conseguenza, la prima opzione è l'unica che ricostruisce correttamente la logica senza rami del compilatore.

2 ARM

Esercizio 7

2.75 punti

Quale è una caratteristica distintiva dell'indirizzamento in ARM?

- a) In ARM, l'indirizzamento basato su registri consente di accedere alla memoria utilizzando solo 2 argomenti: Base e Displacement, con l'indirizzo calcolato come $Base + Displacement$.

- b) In ARM, l'indirizzamento basato su registri consente di accedere alla memoria utilizzando 2 o 3 argomenti: Base e Index, con l'indirizzo calcolato come $\text{Base} + \text{Index} * \text{Scale}$, oppure Base, Offset, con l'indirizzo calcolato come $\text{Base} + \text{Offset}$.
- c) In ARM, l'indirizzamento basato su registri consente di accedere alla memoria utilizzando 3 argomenti: Base, Index e Scale, con l'indirizzo calcolato come $\text{Base} + \text{Index} * \text{Scale}$.
- d) In ARM, l'indirizzamento basato su registri consente di accedere alla memoria utilizzando 3/4 argomenti: Base, Index, Scale e Displacement, con l'indirizzo calcolato come $\text{Base} + (\text{Index} * \text{Scale}) + \text{Displacement}$.

Soluzione: In ARM, l'indirizzamento basato su registri consente di accedere alla memoria utilizzando 2 o 3 argomenti. Le modalità di indirizzamento più comuni sono:

- Base + Offset: L'indirizzo viene calcolato come $\text{Base} + \text{Offset}$, dove l'Offset è una costante che viene aggiunta al registro di base.
- Base + Index * Scale: L'indirizzo viene calcolato come $\text{Base} + (\text{Index} * \text{Scale})$, dove l'Index è un registro che contiene un indice e la Scale è un fattore di scala (1, 2, 4 o 8) che moltiplica l'Index.

La distinzione tra queste modalità viene fatta (e non viene fatto semplicemente $\text{Base} + \text{Index} * 1$) in quanto l'offset è una costante a 12 bit, mentre l'indice è un registro che può essere moltiplicato per un fattore di scala. Di conseguenza, la seconda opzione è corretta.

Esercizio 8

2.75 punti

L'istruzione di caricamento multiplo in ARM consente di caricare più registri da memoria in un'unica istruzione. Quale è la sintassi corretta per questa istruzione e come viene specificato l'elenco dei registri da caricare?

- a) L'istruzione di caricamento multiplo in ARM utilizza la sintassi: `LDM{cond} Rn!, {Rlist}`, dove Rn è il registro di base, Rlist è una lista di registri racchiusa tra parentesi graffe {} che specifica i registri da caricare, e il punto esclamativo (!) indica che il registro di base viene aggiornato dopo il caricamento.
- b) L'istruzione di caricamento multiplo in ARM utilizza la sintassi: `LDM{cond} Rn, {Rlist}`, dove Rn è il registro di base, Rlist è una lista di registri racchiusa tra parentesi graffe {} che specifica i registri da caricare, e il punto esclamativo (!) indica che il registro di base non viene aggiornato dopo il caricamento.
- c) L'istruzione di caricamento multiplo in ARM utilizza la sintassi: `LDM{cond} Rn!, {Rlist}`, dove Rn è il registro di base, Rlist è una lista di registri racchiusa tra parentesi graffe {} che specifica i registri da caricare, e il punto esclamativo (!) indica che il registro di base non viene aggiornato dopo il caricamento.
- d) L'istruzione di caricamento multiplo in ARM utilizza la sintassi: `LDM{cond} Rn, {Rlist}`, dove Rn è il registro di base, Rlist è una lista di registri racchiusa tra parentesi graffe {} che specifica i registri da caricare, e il punto esclamativo (!) indica che il registro di base viene aggiornato dopo il caricamento.

Soluzione: L'istruzione di caricamento multiplo in ARM utilizza la sintassi: `LDM{cond} Rn!, {Rlist}`, dove Rn è il registro di base, Rlist è una lista di registri racchiusa tra parentesi graffe {}

{ } che specifica i registri da caricare, e il punto esclamativo (!) indica che il registro di base viene aggiornato dopo il caricamento. Questa modalità è nota come "post-indexed" e consente di caricare i registri specificati e poi aggiornare il registro di base con l'offset totale dei dati caricati. Di conseguenza, la prima opzione è corretta.

Esercizio 9

2.75 punti

Esiste una procedura generica "multAccess" che esegue delle operazioni di lettura e scrittura su più vettori di interi, questa a sua volta per semplificare i calcoli al suo interno invoca una tal funzione "calc" che a partire dal vettore in origine, la dimensione del vettore e un indice restituisce un intero, questa funzione è la seguente

```
1 int calc(int a[], int i, int n) {  
2     return a[i] + a[n/2+i];  
3 }
```

Considerando che la funzione "multAccess" invoca la funzione "calc" rispettando le convenzioni di chiamata, quali sono le istruzioni in ARM mancanti nella seguente porzione di codice che rappresenta la funzione "multAccess"?

```
1 calc:  
2     add    r2, r2, r2, lsr #31  
3     add    r2, r1, r2, asr #1  
4     ldr    r2, [r0, r2, lsl #2]  
5     ...  
6     add    r0, r0, r2  
7     bx     lr
```

- a) ldr r0, [r2, r1, lsl #2]
- b) ldr r1, [r2, r1, lsl #2]
- c) ldr r3, [r0, r1, lsl #2]
- d) ldr r0, [r2, r1, lsl #2]
- e) Nessuna delle precedenti

Soluzione: Questa semplice funzione calcola la somma di due elementi di un array, uno all'indice i e l'altro all'indice $n/2 + i$. Per rispettare le convenzioni di chiamata in ARM, i parametri della funzione sono passati nei registri: a in $r0$, i in $r1$ e n in $r2$. Il risultato della funzione deve essere restituito in $r0$.

Noriamo che le prime due istruzioni eseguono la divisione intera $n/2$ in modo ottimizzato, di fatto l'istruzione `add r2, r2, r2, lsr #31` aggiunge eventualmente 1 a n se questo è negativo, mentre l'istruzione `add r2, r1, r2, asr #1` esegue la divisione aritmetica per 2 di n e somma il risultato a i , ottenendo così l'indice $n/2 + i$.

Poi viene caricato in $r2$ il valore di $a[n/2 + i]$ con l'istruzione `ldr r2, [r0, r2, lsl #2]`, che utilizza l'indice calcolato in $r2$ per accedere all'array a (in $r0$), moltiplicando l'indice per 4 (dato che ogni elemento è un intero a 32 bit) con lo shift logico a sinistra.

Notiamo dall'ultima istruzione che la somma viene correttamente restituita da $r0$ e che il valore di $a[i]$ non è ancora stato caricato, inoltre questo deve per forza di cose essere in $r0$ in quanto non vengono usati altri registri al di fuori di $r0$ e $r2$ e $r2$ è già occupato dal valore di $a[n/2 + i]$, l'indice da usare è memorizzato in $r1$ per *calling convention* e l'indirizzo di a è memorizzato in $r0$, dunque l'istruzione mancante è `ldr r0, [r0, r1, lsl #2]`, che carica in $r0$ il valore di $a[i]$ già in $r0$ e con l'indice i in $r1$ moltiplicato per 4 con lo shift logico a sinistra. Dopo questa istruzione, $r0$ contiene il valore di $a[i]$, $r2$ contiene il valore di $a[n/2 + i]$,

e l'istruzione `add r0, r0, r2` somma questi due valori e memorizza il risultato in `r0`, che viene poi restituito al chiamante con `bx lr`.

Notiamo come nessuna delle altre opzioni proposte è corretta, quindi la risposta giusta è e).

Esercizio 10

2.75 punti

Viene data la seguente funzione in C:

```
1 int mystery(int a[], int n, int b[], int m){
2     int res=0;
3     for(int i=0;i<n && (res<a[i]||res<b[i]) && i < m;i++){
4         res += a[i] + b[i/2];
5         a[i] = b[i];
6     }
7     return res;
8 }
```

Quale di queste istruzioni è corretta per completare la traduzione del seguente frammento di codice in ARM:

```
1 mystery:
2     push    {r4, r5, r6, r7, lr}
3     subs    r5, r1, #0
4     ble     .L6
5     mov     r6, r2
6     mov     r7, r3
7     sub     lr, r0, #4
8     subs    r2, r2, #4
9     mov     ip, #0
10    mov     r0, ip
11    b       .L3
12 .L4:
13    X1
14    add     r3, ip, ip, lsr #31
15    asrs    r3, r3, #1
16    ldr     r3, [r6, r3, lsl #2]
17    add     r1, r1, r3
18    add     r0, r0, r1
19    ldr     r1, [r2, #4]!
20    X2
21    add     ip, ip, #1
22    cmp     r5, ip
23    beq     .L1
24 .L3:
25    add     lr, lr, #4
26    mov     r4, lr
27    ldr     r1, [lr]
28    cmp     r1, r0
29    bgt     .L4
30    ldr     r3, [r2, #4]
31    cmp     r3, r0
32    bgt     .L4
33 .L1:
```

```

34         pop        {r4, r5, r6, r7, pc}
35 .L6:
36         movs       r0, #0
37         b          .L1

```

- a) X1: `cmp r7, ip; bge .L1` X2: `str r0, [r4]`
- b) X1: `cmp r7, ip; ble .L1` X2: `str r1, [r6]`
- c) X1: `cmp r7, ip; ble .L1` X2: `str r1, [r4]`
- d) X1: `cmp r7, r0; blt .L1` X2: `str r1, [r2]`
- e) Nessuna delle precedenti

Soluzione: Analizziamo il codice; per le convenzioni di chiamata i parametri sono passati nei registri `r0` (a), `r1` (n), `r2` (b) e `r3` (m). Notiamo come vengano preservati i valori di `r4`, `r5`, `r6`, `r7`, `lr` con l'istruzione `push`. Viene poi controllato che il registro `r1` (n) sia maggiore di 0, altrimenti viene saltata tutta la funzione e restituito direttamente 0. L'indirizzo di base di `b` viene salvato in `r6` e il valore di `m` viene salvato in `r7`. Viene poi per qualche ragione sottratto 4 all'indirizzo di `a` e salvato in `lr`, stessa cosa con `r2` (b) a cui viene sottratto 4. L'*index pointer* viene inizializzato a 0 in `ip` e viene anche inizializzato a 0 il registro `r0` che assumiamo per ora che conterrà il valore di `res` (dato che alla fine della funzione questo viene restituito in `r0` per convenzioni di chiamata). Viene poi eseguito un salto incondizionato a `.L3`.

Seguiamo il controllo del codice all'interno del ciclo `.L3`; la prima cosa che viene fatta è l'incremento del puntatore `lr` di 4, questo porta `lr` a contenere l'elemento del vettore `a` che si sta analizzando in questo momento, viene poi caricato in `r1` il valore di `a[i]` con l'istruzione `ldr r1, [lr]`, e questo viene confrontato con `r0` (`res`) per verificare la condizione `res < a[i]`. Se questa è vera non sono necessarie altre verifiche e si salta direttamente al blocco `.L4` che implementa il corpo del ciclo, altrimenti viene caricato in `r3` il valore di `b[i/2]` con l'istruzione `ldr r3, [r2, #4]`, e questo viene confrontato con `r0` (`res`) per verificare la condizione `res < b[i]`, in questo caso si usa l'offset perchè non è stato ancora incrementato il puntatore `r2` (b) e quindi questo punta a `b[i - 1]` e con l'offset di 4 si accede a `b[i]`. Se anche questa condizione è falsa, allora si passa al blocco `.L1` che è il blocco di uscita dal ciclo, altrimenti si entra nel blocco `.L4` che implementa il corpo del ciclo.

Prima di analizzare il blocco `.L4-.L1`, ricapitoliamo ciò che ci aspettiamo di vedere all'interno del ciclo:

- Caricamento di qualche registro con il valore di `a[i]`. (Già fatto nel blocco `.L3` in `r1`)
- Caricamento di qualche registro con il valore di `b[i/2]`. Da fare!
- Caricamento di qualche registro con il valore di `b[i]`. Da fare in quanto potrebbe non essere stato eseguito nel blocco `.L3` se la condizione `res < a[i]` è vera.
- Calcolo di `res += a[i] + b[i/2]`. Da fare!
- Salvataggio di `b[i]` in `a[i]`. Da fare!
- Verifica della condizioni di continuazione del ciclo $i < m$ e $i < n$

- Verifica delle condizioni di continuazione del ciclo $res < a[i]$ e $res < b[i]$. Già fatto nel blocco .L3 con i confronti e i salti condizionali.

Ora notiamo che le prime due istruzioni nel blocco L4 siano ancora da completare, le tralasciamo per un momento.

Successivamente notiamo che viene eseguita la divisione intera $i/2$ e questa viene arrotondata per difetto. Questo viene immediatamente usato e sovrascritto per caricare in **r3** il valore di $b[i/2]$ con l'istruzione `ldr r3, [r6, r3, lsl #2]`. Viene quindi sommato a **r1** ($a[i]$) il valore di **r3** ($b[i/2]$) e il risultato viene memorizzato in **r1**, immediatamente sommato a **r0** (res) e memorizzato in **r0** (res). Viene poi sovrascritto il registro con il valore di $b[i]$ con l'istruzione `ldr r1, [r2, #4]!`, che carica in **r1** il valore di $b[i]$ e poi incrementa il puntatore **r2** (b) di 4, portandolo a puntare a $b[i + 1]$ già preparandolo alla prossima iterazione.

Ora notiamo che manca una istruzione, in quanto nessuna delle successive istruzioni esegue il salvataggio di $b[i]$ in $a[i]$ (infatti viene eseguito il solo incrementando dell'indice **ip** e la verifica che sia verificata la condizione $i < n$), questo è un passaggio necessario all'interno del ciclo, e l'unica istruzione che esegue un'operazione di scrittura in memoria è l'istruzione **str r1, [r6]**, che salva il valore di **r1** ($b[i]$) all'indirizzo puntato da **r6** (a), dunque questa è l'istruzione mancante.

Quindi l'ultima cosa che rimane da fare che ancora non è stata fatta è la verifica della condizione $i < m$ che è una delle condizioni di continuazione del ciclo, questa condizione viene verificata con l'istruzione `cmp r7, ip; ble .L1`, che confronta il valore di m (in **r7**) con l'indice i (in **ip**) e se $i \geq m$ salta al blocco di uscita dal ciclo .L1.

Di conseguenza, la risposta corretta è b)

Notare come al termine della funzione per ritornare al chiamante viene usata l'istruzione `pop {r4, r5, r6, r7, pc}`, che oltre a ripristinare i registri preservati con il `push` all'inizio della funzione, carica anche il Program Counter (PC) con il valore salvato nello stack al momento del `push`, che è l'indirizzo di ritorno al chiamante. Questa è una tecnica comune in ARM per gestire il ritorno da una funzione, e permette di evitare l'uso esplicito dell'istruzione `bx lr` per il ritorno, semplificando così la gestione del flusso di controllo.

Esercizio 11

2.75 punti

Considerando la seguente funzione in C:

```
1 void mystery(int a[], int n, int b[], int m, int* c){
2     for(short i=0; i<a[i]&& i<b[i]; i++){
3         if(a[i]>b[i])
4             *(c++) = *(a++) - *(b++);
5         else
6             *(c++) = *(b++) - *(a++);
7     }
8 }
```

Quale di queste istruzioni è corretta per completare la traduzione del seguente frammento di codice in ARM:

```
1 mystery:
2     ldr    r1, [r0]
3     cmp    r1, #0
4     ble    .L8
5     push   {r4, r5, r6, lr}
6     add    lr, r0, #4
```

```

7      ldr      r4, [sp, #16]
8      adds    r4, r4, #4
9      movs    r3, #0
10     mov     r0, r3
11     mov     ip, r3
12 .L3:
13     mov     r6, r2
14     ldr     r0, [r2, r0]
15     X1
16     ble     .L1
17     cmp     r0, r1
18     ittee   lt
19     ldrlt   r1, [lr, #-4]
20     ldrlt   r0, [r2]
21     ldrge   r1, [r6]
22     ldrge   r0, [lr, #-4]
23     X2
24     str     r1, [r4, #-4]
25     adds    r3, r3, #1
26     sxth    r3, r3
27     mov     ip, r3
28     lsls    r0, r3, #2
29     ldr     r1, [lr, r3, lsl #2]
30     adds    r2, r2, #4
31     add     lr, lr, #4
32     adds    r4, r4, #4
33     cmp     r3, r1
34     blt     .L3
35 .L1:
36     pop     {r4, r5, r6, pc}
37 .L8:
38     bx      lr

```

Quale delle seguenti opzioni è corretta per le etichette X1 e X2?

- a) X1: `cmp r3, r1`; X2: `sub r1, r1, r0`
- b) X1: `cmp r0, r3`; X2: `add r1, r1, r0`
- c) X1: `cmp r0, r3`; X2: `sub r1, r1, r0`
- d) X1: `cmp r0, r1`; X2: `sub r0, r0, r1`
- e) Nessuna delle precedenti

Soluzione: Analizziamo il flusso del ciclo `for`. La condizione di permanenza è doppia: $i < a[i]$ e $i < b[i]$. Nel codice ARM, `r3` rappresenta l'indice `i`.

Prima della label `.L3`, `r1` viene caricato con `a[i]` (inizialmente `a[0]`). All'interno del ciclo, l'istruzione `ldr r0, [r2, r0]` carica in `r0` il valore di `b[i]` (poiché `r2` è la base di `b` e l'offset iniziale è 0). Di conseguenza, **X1** deve verificare la condizione $i < b[i]$, ovvero `cmp r0, r3`. Se $i \geq b[i]$ (`ble`), il ciclo termina saltando a `.L1`.

Successivamente, il codice deve calcolare la differenza assoluta tra `a[i]` e `b[i]`. L'istruzione `cmp r0, r1` confronta `b[i]` e `a[i]`. Il blocco `ITTEE` carica i valori nei registri `r1` e `r0` in modo che il minuendo sia sempre il maggiore e il sottraendo il minore. **X2** completa l'operazione

aritmetica con `sub r1, r1, r0`, il cui risultato viene poi memorizzato nel puntatore `c` tramite `str r1, [r4, #-4]`.

Risposta giusta: c)

Esercizio 12

2.75 punti

Viene fornita la seguente funzione in C:

```
1 void copia_stringa(char d[], char s[]) {
2     int i = 0;
3     while (1) {
4         char c = s[i];
5         d[i] = c;
6         if (c == '\0') break;
7         i++;
8     }
9 }
```

Quale di queste istruzioni è corretta per completare la traduzione del seguente frammento di codice in ARM:

```
1 copia_stringa:
2     ldrb    r3, [r1]
3     strb    r3, [r0]
4     cbz     r3, .L1
5 .L3:
6     X1
7     cmp     r3, #0
8     bne     .L3
9 .L1:
10    bx      lr
```

Quale delle seguenti opzioni è corretta per le etichette X1 e X2?

- a) `ldrb r3, [r1], #1 ; strb r3, [r0], #1`
- b) `ldrb r3, [r1, #1]! ; strb r3, [r0, #1]!`
- c) `ldrb r3, [r1, r2] ; strb r3, [r0, r2]`
- d) `ldrb r3, [r1, #4]! ; strb r3, [r0, #4]!`
- e) Nessuna delle precedenti

Soluzione: La funzione C implementa una copia di stringa carattere per carattere. In ARM, per ottimizzare il ciclo `while`, il compilatore utilizza l'indirizzamento con *write-back*.

Il primo carattere viene gestito fuori dal ciclo per verificare immediatamente se la stringa è vuota (`cbz r3, .L1`). All'interno del ciclo `.L3`, dobbiamo avanzare gli indici di sorgente (`r1`) e destinazione (`r0`) e copiare il carattere successivo. L'istruzione `ldrb r3, [r1, #1]!` carica il byte all'indirizzo `r1 + 1` e aggiorna contemporaneamente il registro `r1` (pre-incremento). Lo stesso avviene per la memorizzazione con `strb r3, [r0, #1]!`. Poiché stiamo lavorando con un array di `char`, l'incremento corretto è di 1 byte. L'opzione con l'offset `#4` sarebbe stata corretta per un array di `int`.

Risposta giusta: b)